

Week 4 Stack Exercises

These exercises apply stacks to browser history and delimiter checking. Use the `Stack` class developed in class.

Learning goals

By the end of these exercises, you should be able to:

- use `push`, `pop`, `peek` and `is_empty`
- apply a stack to a practical problem
- use a stack to process nested data

Exercise 1: Browser back history

Create a small browser-history program. The program should remember previously visited pages and return to them in reverse order.

Required behaviour

- The browser begins on `Home`.
- Visiting a new page saves the current page in the history stack.
- Going back removes the most recent page from the history stack.
- Going back when the history is empty leaves the current page unchanged.

Starter code

```
history = Stack()
current_page = "Home"

def visit(page):
    global current_page

    # Save the page that the user is leaving.
    # Change current_page to the new page.
    pass

def go_back():
    global current_page

    # Check whether an earlier page is available.
    # If it is, remove it from history and make it current.
    # Return the current page.
    pass
```

Test code

```
print(current_page)

visit("Products")
visit("Basket")
visit("Checkout")

print(current_page)
print(go_back())
print(go_back())
print(go_back())
print(go_back())
```

Expected output:

```
Home
Checkout
Basket
Products
Home
Home
```

The final call cannot go any further back, so the browser remains on **Home**.

Code hints

- `visit()` should push the old value of `current_page` before changing it.
- `go_back()` should call `is_empty()` before calling `pop()`.
- The value returned by `pop()` becomes the new `current_page`.
- If the stack is empty, return `current_page` without changing it.
- Trace `current_page` and the history stack separately while testing.

Pseudocode:

```
VISIT new_page
    PUSH current_page onto history
    SET current_page to new_page

GO_BACK
    IF history is not empty
        SET current_page to POP from history
    RETURN current_page
```

Questions

1. Why is the most recently visited page the first page returned by `go_back()`?
2. What information is stored in the stack?

3. What information is stored in `current_page`?
4. Why must `go_back()` check `is_empty()` before calling `pop()`?
5. What additional stack would be needed to support a Forward button?

Reference examples

Try the exercise before consulting these examples:

- [W3Schools: Stacks with Python](#)
- [GeeksforGeeks: Implementing Backward and Forward Buttons of a Browser](#)
- [Python documentation: Using Lists as Stacks](#)

Exercise 2: Balanced delimiters

Programs and configuration files often contain nested brackets. A stack can check whether opening and closing delimiters match correctly.

Complete the function using your `Stack` class.

Starter code

```
def delimiters_are_balanced(text):
    stack = Stack()

    matching = {
        ")": "(",
        "]": "[",
        "}": "{",
    }

    for character in text:
        if character in "([{":
            # Push the opening delimiter.
            pass

        elif character in matching:
            # Reject a closing delimiter when the stack is empty.
            # Pop the most recent opening delimiter.
            # Check whether the pair matches.
            pass

    # The stack must be empty when processing finishes.
    return False
```

Test code

```
tests = [
    "print(items[0])",
    "calculate((4 + 5) * 2)",
```

```
"colours = {red: [255, 0, 0]}",
"print(items[0]",
"calculate((4 + 5)",
"]unexpected["
]

for test in tests:
    print(delimiters_are_balanced(test), test)
```

Expected output:

```
True print(items[0])
True calculate((4 + 5) * 2)
True colours = {red: [255, 0, 0]}
False print(items[0]
False calculate((4 + 5)
False ]unexpected[
```

Code hints

- Push only opening delimiters: (, [and {.
- Each closing delimiter is a key in the `matching` dictionary.
- `matching[character]` gives the opening delimiter expected at the top of the stack.
- A closing delimiter cannot match when the stack is empty.
- Save the value returned by `pop()` and compare it with `matching[character]`.
- Stop immediately and return `False` when a mismatch is found.
- Opening delimiters may remain after the loop finishes.
- `return stack.is_empty()` may be useful as the final line.

Pseudocode:

```
CREATE an empty stack

FOR each character in the text
    IF character is an opening delimiter
        PUSH character
    ELSE IF character is a closing delimiter
        IF stack is empty
            RETURN false
        SET opening to POP from stack
        IF opening does not match character
            RETURN false

RETURN whether the stack is empty
```

Further challenge

Change the function so that it returns a result and an explanation:

```
(True, "Valid")  
(False, "Unexpected closing delimiter")  
(False, "Mismatched delimiter")  
(False, "Unclosed delimiter")
```

Reference examples

Try the exercise before consulting these examples:

- [GeeksforGeeks: Check for Balanced Parentheses in Python](#)
- [GeeksforGeeks: Balanced Brackets Using a Stack](#)
- [W3Schools: Stacks with Python](#)